

4.2. Numpy

Tema 4. Implementación y aplicación

Javier Paris

Table of contents

1	Introducción a Numpy	2
1.1	¿Qué es Numpy?	2
1.2	Instalación	2
1.3	Importar Numpy	3
2	Creación de arrays y matrices	3
2.1	Creación de arrays y matrices	3
2.2	Creación de arrays a partir de listas	3
2.3	Creación de arrays a partir iterables	3
2.4	Creación de matrices a partir de listas	4
3	Exploración de arrays y matrices	4
3.1	Propiedades de los arrays	4
3.2	Acceso a elementos	5
3.3	Operaciones aritméticas	5
4	Funciones	5
4.1	Funciones matemáticas	5
4.2	Operaciones vectoriales	6
4.3	Operaciones matriciales	6
4.4	Operaciones estadísticas	7
5	Conclusión	8
5.1	Conclusiones	8

1 Introducción a Numpy

1.1 ¿Qué es Numpy?

Numpy es una librería de Python que proporciona soporte para arrays y matrices multidimensionales, junto con una colección de funciones matemáticas para operar con estos arrays.

- Permite realizar operaciones matemáticas y lógicas sobre arrays de forma simple y eficiente.
- Proporciona una gran cantidad de funciones matemáticas para operar con arrays y matrices.

<https://numpy.org/>



1.2 Instalación

Por defecto, numpy ya viene instalado en diversos entornos de desarrollo de Python, como Anaconda o Jupyter Notebook (Google Colab).

Para instalar numpy en tu entorno local, puedes hacerlo mediante pip:

```
pip install numpy
```

o desde un cuaderno de Jupyter:

```
!pip install numpy
```

1.3 Importar Numpy

Como cualquier librería en Python, primero debemos importarla para poder utilizarla. Es muy común importar numpy con el alias `np` (pero no es obligatorio):

```
import numpy as np
```

2 Creación de arrays y matrices

2.1 Creación de arrays y matrices

Podemos crear arrays y matrices de numpy de varias formas:

- A partir de listas o tuplas
- Utilizando funciones específicas de numpy
- A partir de un rango de valores
- etc.

2.2 Creación de arrays a partir de listas

Podemos crear un array de numpy a partir de una lista de Python:

```
l = [1, 2, 3, 4, 5]  
a = np.array(l)  
print(a)
```

```
[1 2 3 4 5]
```

2.3 Creación de arrays a partir iterables

Podemos crear un array de numpy a partir de cualquier iterable:

```
it = range(0, 10, 2)  
a = np.array(it)  
print(a)
```

```
[0 2 4 6 8]
```

2.4 Creación de matrices a partir de listas

Podemos crear una matriz de numpy a partir de una lista de listas:

```
l = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
m = np.array(l)
print(m)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

3 Exploración de arrays y matrices

3.1 Propiedades de los arrays

Los arrays de numpy tienen varias propiedades que nos permiten conocer su tamaño, forma, tipo de datos, etc.

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(f"Array:\n{a}\n")
print(f"Shape:\n{a.shape}\n")
print(f"Type:\n{a.dtype}\n")
print(f"Size:\n{a.size}\n")
```

```
Array:
[[1 2 3]
 [4 5 6]]
```

```
Shape:
(2, 3)
```

```
Type:
int64
```

```
Size:
6
```

3.2 Acceso a elementos

Podemos acceder a los elementos de un array de numpy de la misma forma que accedemos a los elementos de una lista de Python. Para acceder a un elemento de una matriz, utilizamos el operador `[]` con una tuple con los índices.

```
a = np.array([1, 2, 3, 4, 5])
print(a[0])
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(a[1, 1])
```

```
1
5
```

3.3 Operaciones aritméticas

Podemos realizar operaciones aritméticas para modificar todos los elementos de un array. Estas operaciones se realizan elemento a elemento.

```
a = np.array([1, 2, 3, 4, 5])
print(a + 1)
b = np.array([5, 4, 3, 2, 1])
print(a + b)
```

```
[2 3 4 5 6]
[6 6 6 6 6]
```

4 Funciones

4.1 Funciones matemáticas

Numpy proporciona una gran cantidad de funciones matemáticas para operar con arrays y matrices:

- `np.sin`, `np.cos`, `np.tan`
- `np.exp`, `np.log`, `np.log10`
- `np.sqrt`, `np.power`, `np.abs`
- `np.mean`, `np.median`, `np.std`
- etc.

4.2 Operaciones vectoriales

Podemos realizar operaciones vectoriales con arrays de numpy:

- Norma: `np.linalg.norm`
- Producto escalar: `@`
- Producto vectorial: `np.dot`

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(f"norm(a):\n{np.linalg.norm(a)}\n")
print(f"scalar:\n{a @ b}\n")
print(f"vectorial:\n{np.dot(a, b)}\n")
```

```
norm(a):
3.7416573867739413
```

```
scalar:
32
```

```
vectorial:
32
```

4.3 Operaciones matriciales

Podemos realizar operaciones matriciales con arrays de numpy:

- Matriz transpuesta: `np.transpose`
- Matriz inversa: `np.linalg.inv`
- Producto matricial: `@`

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(f"transpose(a):\n{np.transpose(a)}\n")
print(f"inverse(a):\n{np.linalg.inv(a)}\n")
print(f"matrix:\n{a @ b}\n")
```

```
transpose(a):
[[1 3]
 [2 4]]
```

```
inverse(a):  
[[-2.   1. ]  
 [ 1.5 -0.5]]
```

```
matrix:  
[[19 22]  
 [43 50]]
```

4.4 Operaciones estadísticas

Podemos realizar operaciones estadísticas con arrays de numpy:

- Media: `np.mean`
- Mediana: `np.median`
- Varianza: `np.var`
- Desviación estándar: `np.std`
 - `ddof=0` para población (por defecto `ddof=1` para muestra)
- Covarianza: `np.cov`
- Correlación: `np.corrcoef`

```
a = np.array([1, 2, 3, 4, 5])  
print(f"mean:\n{np.mean(a)}\n")  
print(f"median:\n{np.median(a)}\n")  
print(f"variance:\n{np.var(a)}\n")  
print(f"std:\n{np.std(a)}\n")
```

```
mean:  
3.0
```

```
median:  
3.0
```

```
variance:  
2.0
```

```
std:  
1.4142135623730951
```

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([5, 4, 3, 2, 1])
print(f"covariance:\n{np.cov(a, b)}\n")
print(f"correlation:\n{np.corrcoef(a, b)}\n")
```

```
covariance:
[[ 2.5 -2.5]
 [-2.5  2.5]]
```

```
correlation:
[[ 1. -1.]
 [-1.  1.]]
```

5 Conclusión

5.1 Conclusiones

- Numpy es una librería de Python que proporciona soporte para arrays y matrices multi-dimensionales.
- Permite realizar operaciones matemáticas sobre arrays de forma simple y eficiente.
- Proporciona una gran cantidad de funciones matemáticas para operar con arrays y matrices.
- Es una librería muy utilizada en el ámbito de la ciencia de datos y el aprendizaje automático.